# CMAI 480

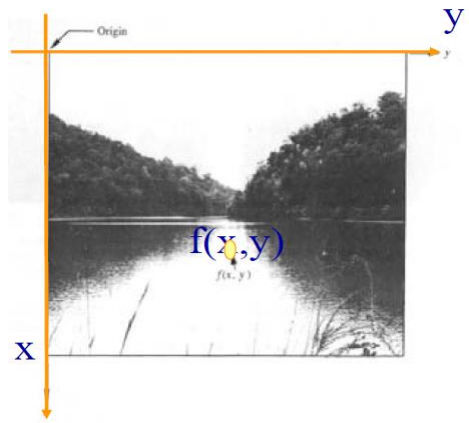
# Computer Vision

Dr. Khalil Ahmad Yousef
Computer Science and Engineering Department
Qatar University

# Image Processing
# Basic Relationship Between Pixels

Reading Assignment: Szeliski, Chapter 3 (Image Processing)

# Neighbors of a pixel




- A pixel **p** at coordinate (*x,y*) has
  - $N_4$(**p**) : 4-neighbors of **p**
    - (x+1, y), (x-1,y),(x,y+1), (x,y-1)

x
x p x
x

  - $N_D$(**p**) : 4-diagonal neighbors of **p**
    - (x+1, y+1), (x+1,y-1),(x-1,y+1), (x-1,y-1)

x x
p
x x

  - $N_8$(**p**) : 8-neighbors of **p** :
    - a combination of $N_4$(**p**) and $N_D$(**p**)

x x x
x p x
x x x

# Distance Measures

- For pixel p, q and z with coordinates (*x,y*), (*s,t*) and (*u,v*) respectively,
- D is a distance function or metric if

a) $D(p,q) \geq 0$ ; (Non-Negative)

b) $D(p,q) = 0$ iff $p=q$ (Identity Element)

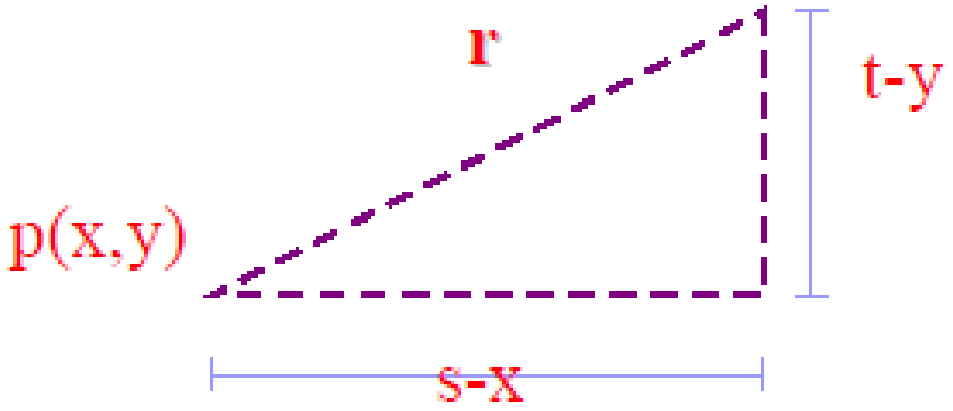
c) $D(p,q) = D(q,p)$ (Symmetry)

d) $D(p,z) \leq D(p,q) + D(q,z)$ (Triangle Inequality)

# Euclidean distance between p and q

$$D_e(p,q) = \left[(x-s)^2 + (y-t)^2\right]^{1/2}$$

q(s,t)




radius (r) centered at (x,y)

# Arithmetic Operators

- Used extensively in most branches of image processing.
- Arithmetic operations between 2 pixels p and q :
  - Addition : p + q used in image average to reduce noise.
  - Subtraction : p - q basic tool in medical imaging.
  - Multiplication : p x q
    - To correct gray-level shading result from nonuniformities in illumination or in the sensor used to acquire the image.
  - Division : p ÷ q
- Arithmetic Operation on entire images are carried out pixel by pixel.

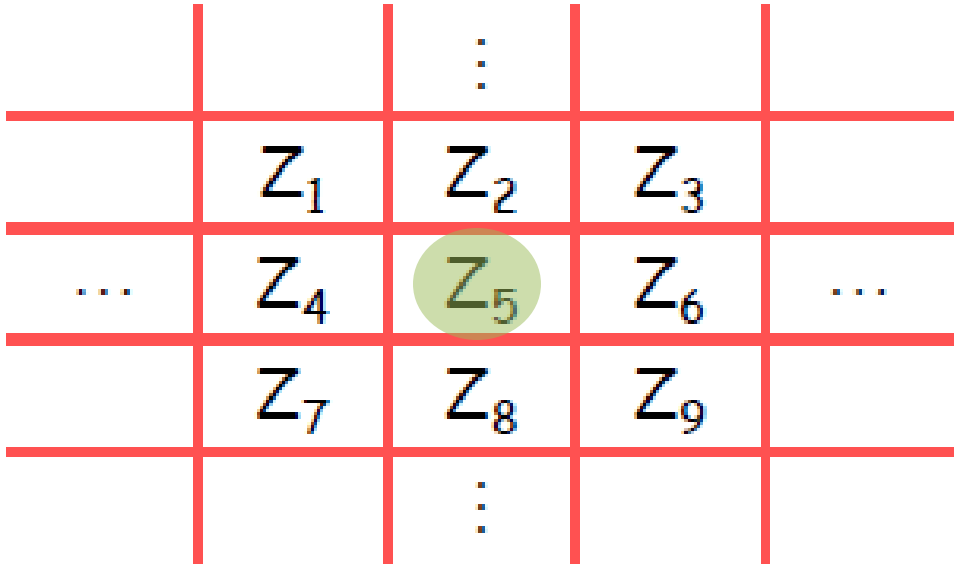
# Logic operations

- AND : p AND q (p•q)
- OR : p OR q (p+q)
- COMPLEMENT : NOT q ($\bar{q}$)
- Logic operations apply only to binary images.
- Arithmetic operations apply to multivalued pixels.
- Logic operations used for tasks such as masking, feature detection, and shape analysis.
- Logic operations perform pixel by pixel.

# IMAGE FILTERING

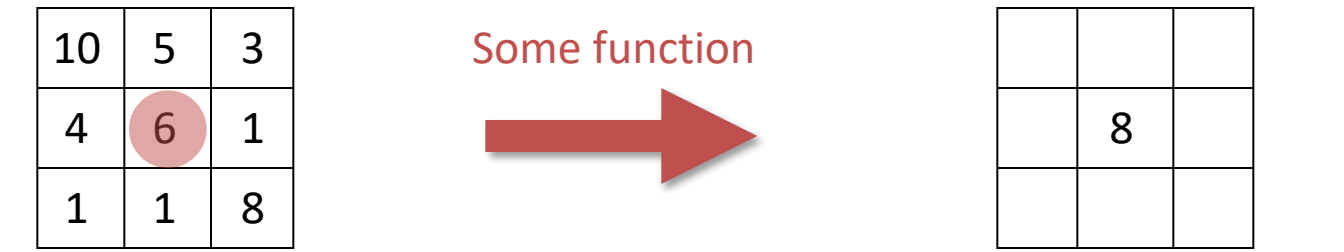
# Image Filtering
## Mask Operation

- Besides pixel-by-pixel processing on entire images, arithmetic and Logical operations are used in neighborhood oriented operations.



# Image filtering

- Modify the pixels in an image based on some function of a local neighborhood of each pixel

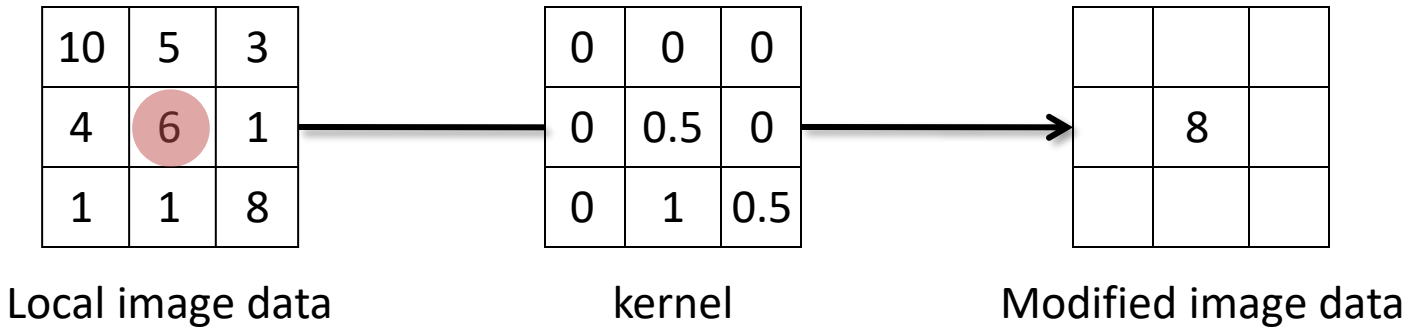



Local image data

Modified image data

# Linear filtering

- One simple version of filtering: linear filtering (cross-correlation, convolution)
  - Replace each pixel by a linear combination (a weighted sum) of its neighbors
- The prescription for the linear combination is called the "kernel" (or "mask", "filter")



# Image Filtering: Why needed?
## Noise reduction

- Given a camera and a still scene, how can you reduce noise?



Take lots of images and average them!

What's the next best thing?

# Mask Operation
## Averaging Principle

- Let the value assigned to a pixel be a function of its gray level and the gray level of its neighbors.
- e.g., replace the gray value of pixel $Z_5$ with the average gray values of it's neighborhood within a 3x3 mask.

$$Z = \frac{1}{9}(Z_1 + Z_2 + Z_3 + \ldots + Z_9)$$

# Mask operator

- In general term:

$$Z = \frac{1}{9}Z_1 + \frac{1}{9}Z_2 + \frac{1}{9}Z_3 + \ldots + \frac{1}{9}Z_9$$

$$= w_1 Z_1 + w_2 Z_2 + w_3 Z_3 + \ldots w_9 Z_9$$

$$= \sum_{i=1}^{9} w_i Z_i$$

| | | |
|---|---|---|
| $w_1$ | $w_2$ | $w_3$ |
| $w_4$ | $w_5$ | $w_6$ |
| $w_7$ | $w_8$ | $w_9$ |



| | | |
|---|---|---|
| 1/9 | 1/9 | 1/9 |
| 1/9 | 1/9 | 1/9 |
| 1/9 | 1/9 | 1/9 |

# Mask coefficient

- Proper selection of the coefficients and application of the mask at each pixel position in an image makes possible a variety of useful image operations
  - noise reduction
  - region thinning
  - edge detection
- Applying a mask at each pixel location in an image is a computationally expensive task and it is performed based on a convolutional operator

# Convolution

$$G[i,j] = \sum_{u=-k}^{k} \sum_{v=-k}^{k} H[u,v]F[i-u, j-v]$$

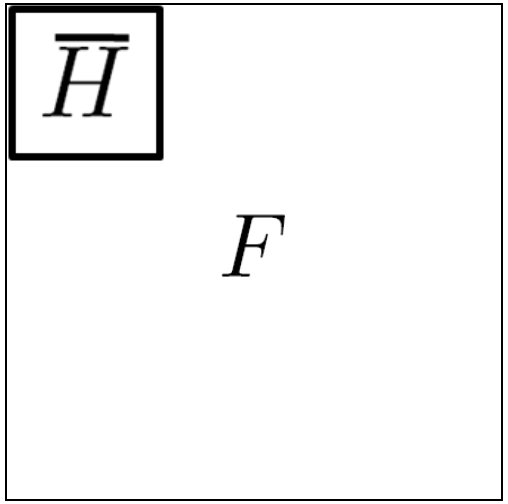
This is called a **convolution** operation

$$G = H * F$$

# Convolution

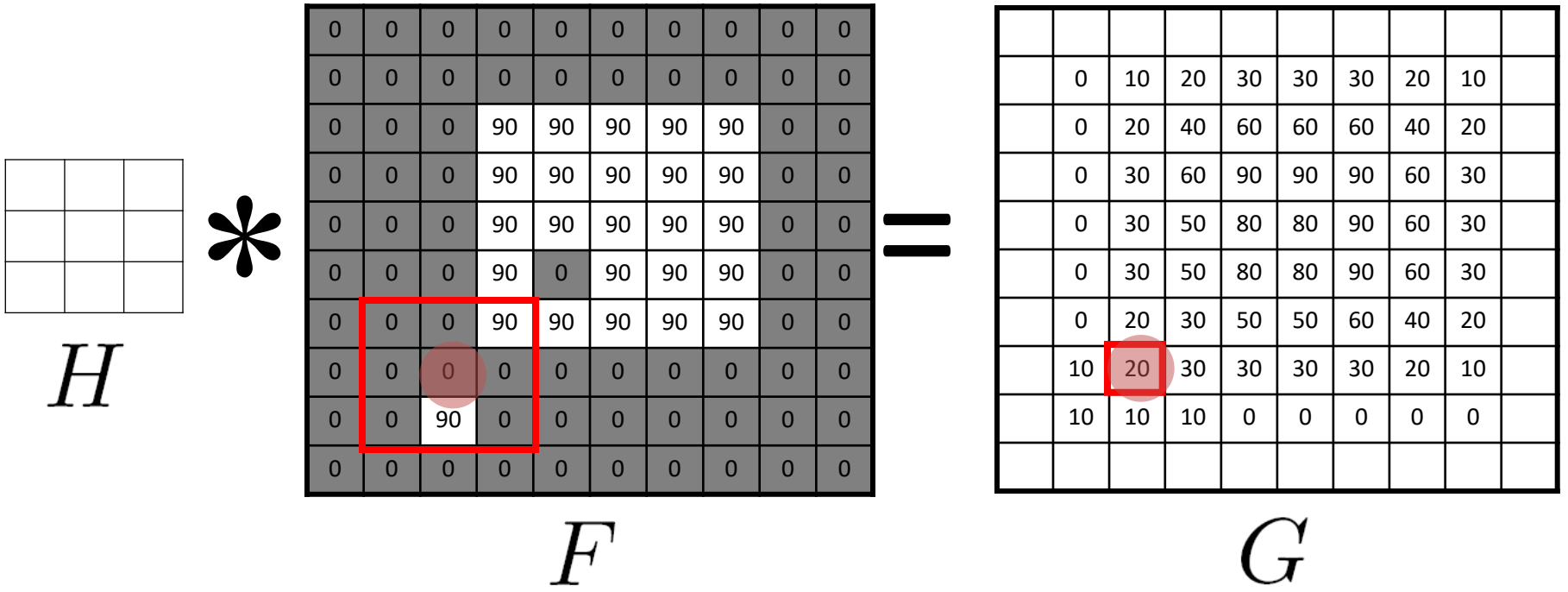



**H (Kernal or Filter)**



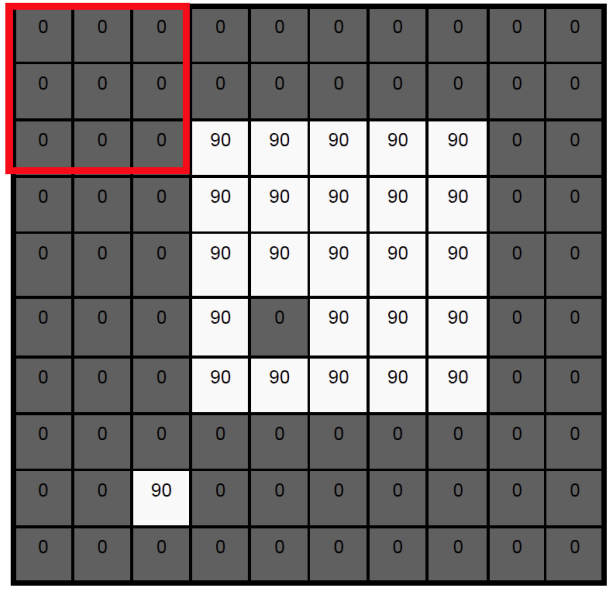

**F (Image)**

# Mean filtering

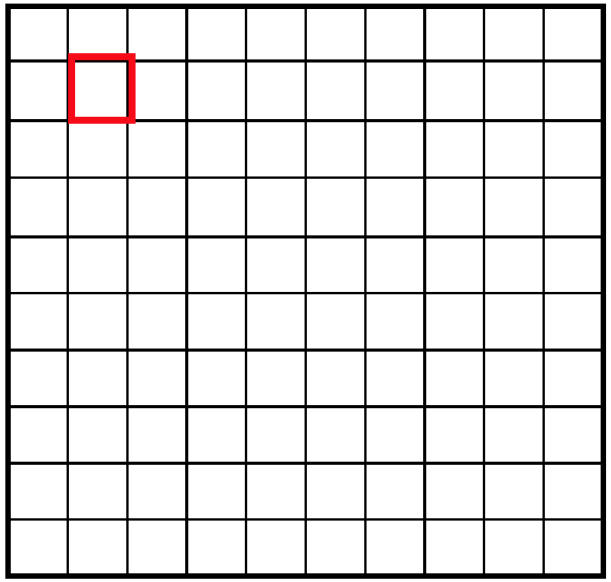


# Mean filtering/Moving average

$F[x, y]$

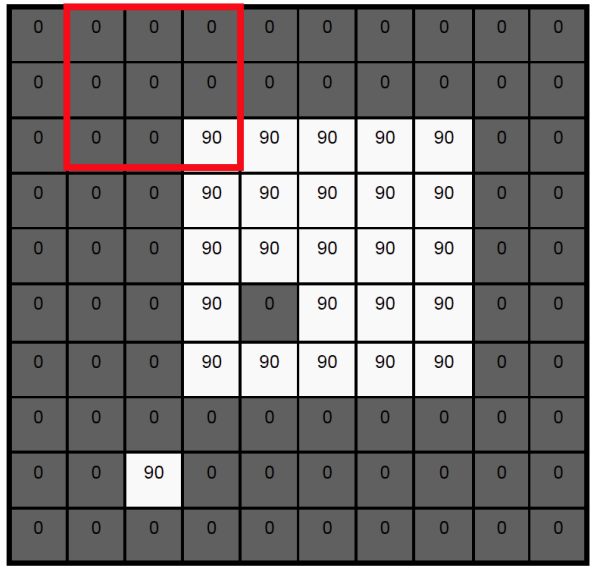



$G[x, y]$

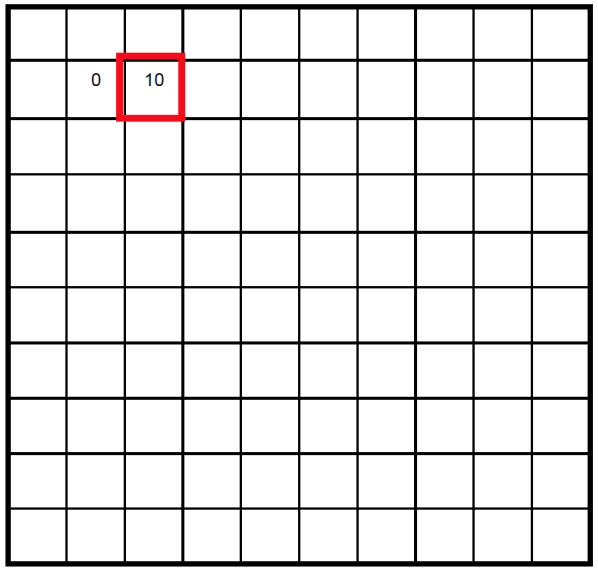


# Mean filtering/Moving average

$F[x, y]$

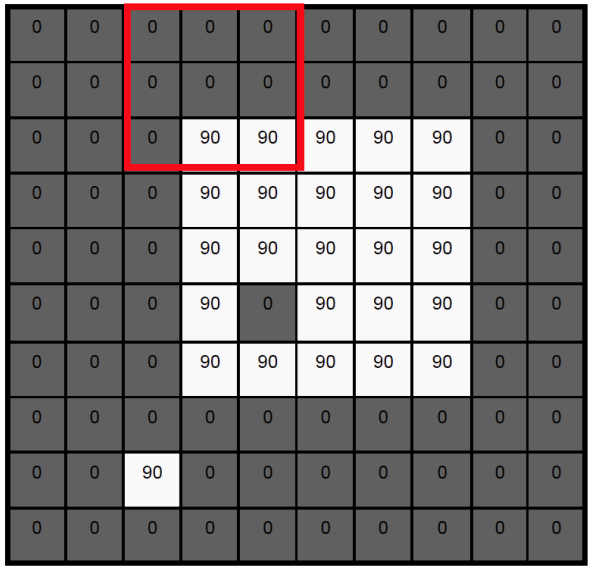



$G[x, y]$

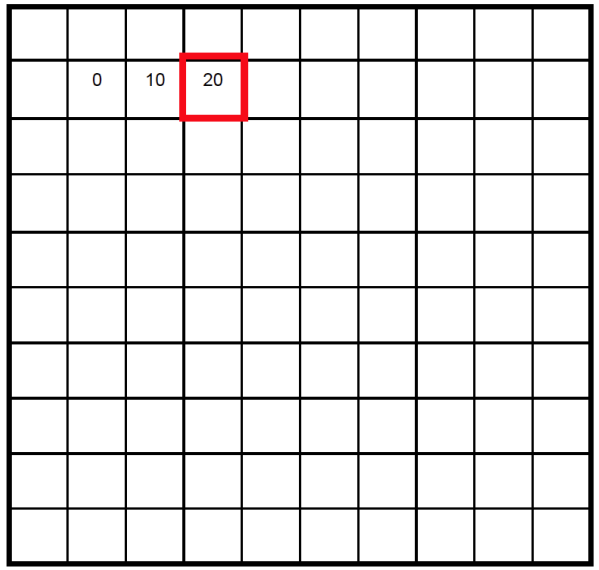


# Mean filtering/Moving average

$F[x, y]$

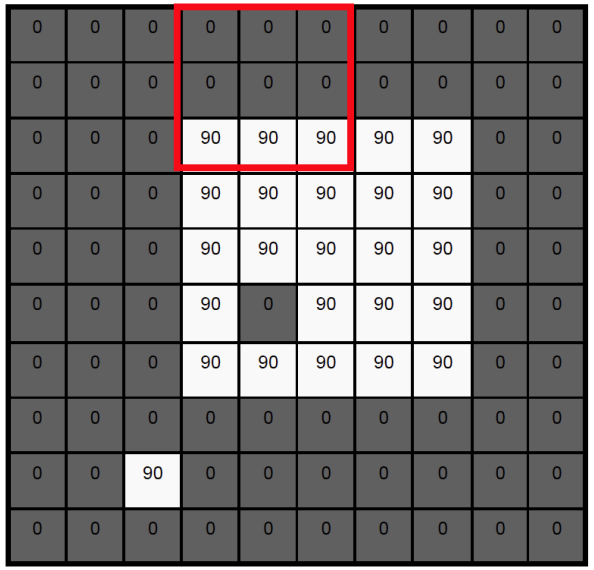



$G[x, y]$

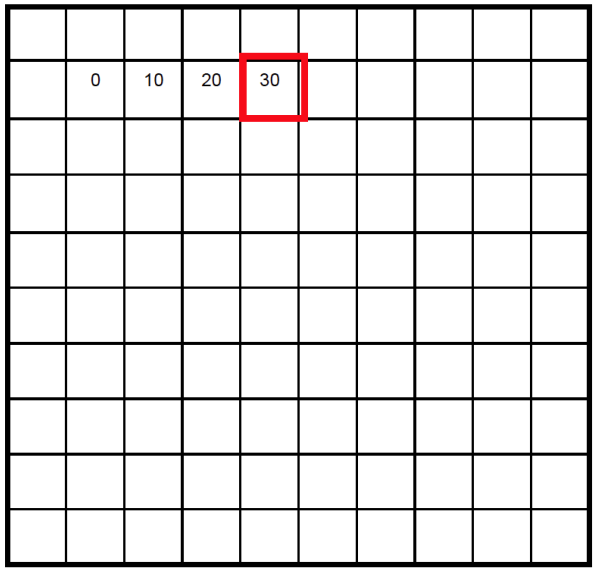


# Mean filtering/Moving average

$F[x, y]$




$G[x, y]$



# Mean filtering/Moving average

$F[x, y]$

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 90 | 90 | 90 | 90 | 90 | 0 | 0 |
| 0 | 0 | 0 | 90 | 90 | 90 | 90 | 90 | 0 | 0 |
| 0 | 0 | 0 | 90 | 90 | 90 | 90 | 90 | 0 | 0 |
| 0 | 0 | 0 | 90 | 0 | 90 | 90 | 90 | 0 | 0 |
| 0 | 0 | 0 | 90 | 90 | 90 | 90 | 90 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 90 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

$G[x, y]$

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | |
| | 0 | 10 | 20 | 30 | 30 | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |

# Mean filtering/Moving average

$F[x, y]$

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 90 | 90 | 90 | 90 | 90 | 0 | 0 |
| 0 | 0 | 0 | 90 | 90 | 90 | 90 | 90 | 0 | 0 |
| 0 | 0 | 0 | 90 | 90 | 90 | 90 | 90 | 0 | 0 |
| 0 | 0 | 0 | 90 | 0 | 90 | 90 | 90 | 0 | 0 |
| 0 | 0 | 0 | 90 | 90 | 90 | 90 | 90 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 90 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

$G[x, y]$

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | |
| | 0 | 10 | 20 | 30 | 30 | 30 | 20 | 10 | |
| | 0 | 20 | 40 | 60 | 60 | 60 | 40 | 20 | |
| | 0 | 30 | 60 | 90 | 90 | 90 | 60 | 30 | |
| | 0 | 30 | 50 | 80 | 80 | 90 | 60 | 30 | |
| | 0 | 30 | 50 | 80 | 80 | 90 | 60 | 30 | |
| | 0 | 20 | 30 | 50 | 50 | 60 | 40 | 20 | |
| | 10 | 20 | 30 | 30 | 30 | 30 | 20 | 10 | |
| | 10 | 10 | 10 | 0 | 0 | 0 | 0 | 0 | |
| | | | | | | | | | |

# Linear filters: examples



Original

*

| | | |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |

# Exercise

**What image operation does filtering with this kernel perform?**

**([0 0 0;**

**0 1 0;**

**0 0 0])**

# Linear filters: examples



Original

$*$

| | | |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |

$=$



Identical image

# The Core Rule

- Because spatial convolution involves a $180°$kernel flip (reflection across both axes), **the spatial direction of the non-zero value in the filter matrix is opposite to the direction the image actually moves.**

- **Right weight $\rightarrow$ Left shift:** Placing a $1$ on the **right** side of the matrix pulls values from the right neighbor, shifting the visual content to the **left**.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \xrightarrow{\text{convolution}} \text{Shifts image LEFT}$$

- **Left weight $\rightarrow$ Right shift:** Placing a $1$ on the **left** side pulls values from the left neighbor, shifting the visual content to the **right**.

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \xrightarrow{\text{convolution}} \text{Shifts image RIGHT}$$

- **Bottom weight $\rightarrow$ Upward shift:** Placing a $1$ at the **bottom** pulls values from below, shifting the image **up**.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \xrightarrow{\text{convolution}} \text{Shifts image UP}$$

- **Top weight $\rightarrow$ Downward shift:** Placing a $1$ at the **top** pulls values from above, shifting the image **down**.

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \xrightarrow{\text{convolution}} \text{Shifts image DOWN}$$

# Exercise

**What image operation does filtering with this kernel perform?**

**([0 0 0;**

**1 0 0;**

**0 0 0])**

# Linear filters: examples



Original

*

| | | |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 0 |
| 0 | 0 | 0 |

# Linear filters: examples

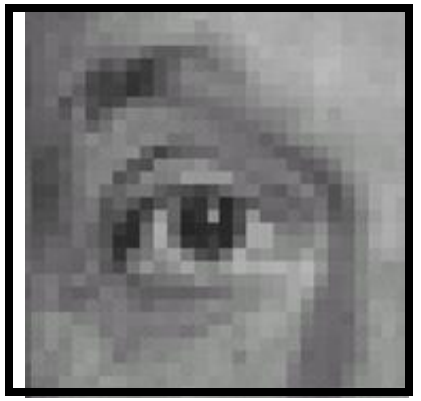


Original

*

| | | |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 0 |
| 0 | 0 | 0 |

=



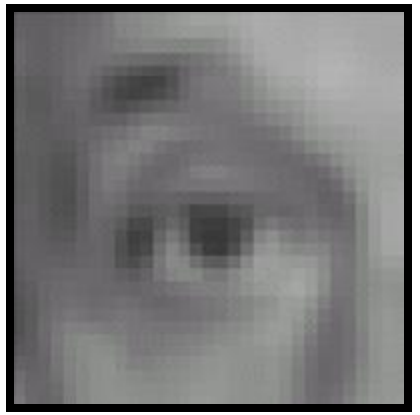
Shifted right by 1 pixel

# Linear filters: examples



Original

$$* \quad \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} =$$



Blur (with a mean filter)

# Linear filters: examples



Original

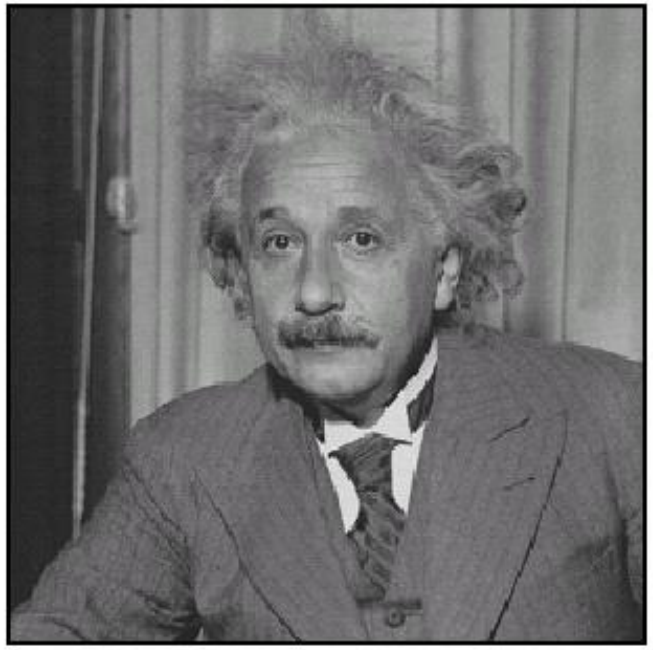
$$*\left(\begin{bmatrix}0&0&0\\0&2&0\\0&0&0\end{bmatrix}-\frac{1}{9}\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}\right)=$$

$$\begin{bmatrix}-1/9&-1/9&-1/9\\-1/9&+17/9&-1/9\\-1/9&-1/9&-1/9\end{bmatrix}$$

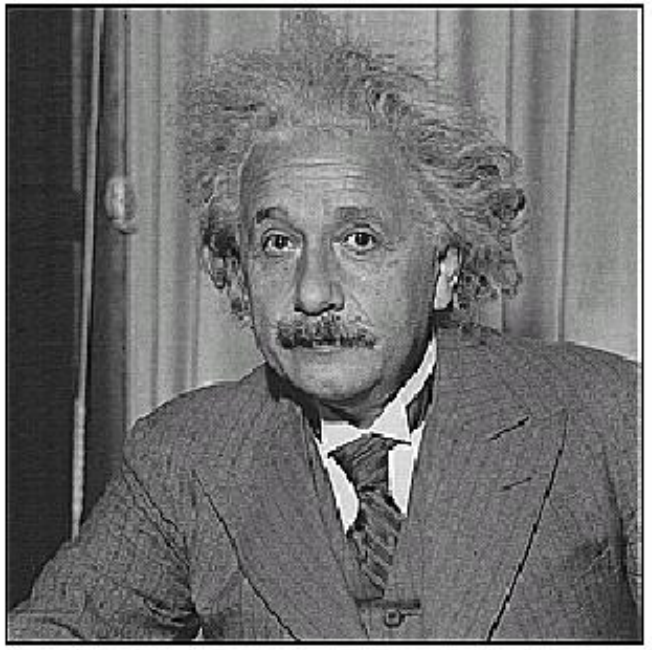


**Sharpening filter**
(accentuates edges)

# Sharpening



before



after

# Sharpening Kernels:

https://en.wikipedia.org/wiki/Kernel_(image_processing)

- Some Examples (usage depends on the application and images at hand):
  - Sobel

$$\frac{1}{8}\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad s_x$$

$$\frac{1}{8}\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \quad s_y$$

**Sum of the sharpening filter needs to be zero. This ensures that the average intensity of the image remains unchanged after filtering**

  - Laplacian

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

# Separable Filter: Main Concept

$\frac{1}{K^2}$

| | | | |
|---|---|---|---|
| 1 | 1 | ⋯ | 1 |
| 1 | 1 | ⋯ | 1 |
| ⋮ | ⋮ | 1 | ⋮ |
| 1 | 1 | ⋯ | 1 |

$\frac{1}{16}$

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

$\frac{1}{256}$

| | | | | |
|---|---|---|---|---|
| 1 | 4 | 6 | 4 | 1 |
| 4 | 16 | 24 | 16 | 4 |
| 6 | 24 | 36 | 24 | 6 |
| 4 | 16 | 24 | 16 | 4 |
| 1 | 4 | 6 | 4 | 1 |

$\frac{1}{8}$

| | | |
|---|---|---|
| −1 | 0 | 1 |
| −2 | 0 | 2 |
| −1 | 0 | 1 |

$\frac{1}{4}$

| | | |
|---|---|---|
| 1 | −2 | 1 |
| −2 | 4 | −2 |
| 1 | −2 | 1 |

$\frac{1}{K}$ | 1 | 1 | ⋯ | 1 |

$\frac{1}{4}$ | 1 | 2 | 1 |

$\frac{1}{16}$ | 1 | 4 | 6 | 4 | 1 |

$\frac{1}{2}$ | −1 | 0 | 1 |

$\frac{1}{2}$ | 1 | −2 | 1 |



(a) box, $K = 5$ (b) bilinear (c) "Gaussian" (d) Sobel (e) corner

**Figure 3.14** *Separable linear filters: For each image (a)–(e), we show the 2D filter kernel (top), the corresponding horizontal 1D kernel (middle), and the filtered image (bottom). The filtered Sobel and corner images are signed, scaled up by 2× and 4×, respectively, and added to a gray offset before display.*

# Separable Filter: Main Concept

- The process of performing a convolution requires $K^2$ (multiply-add) operations per pixel, **where K is the size (width or height)** of the convolution kernel, e.g., the box filter in Figure 3.14a (Previous Slide).
- In many cases, this operation can be:
  - significantly sped up by:
    - first performing a one-dimensional horizontal convolution
    - followed by a one-dimensional vertical convolution,
  - This requires a total of 2K operations per pixel.
  - A convolution kernel for which this is possible is said to be separable.

# Edge Detection

## All about detecting sudden changes in intensity

- Canny Edge Detection
  - Watch the following YouTube video for an example in Python

**OpenCV Python Canny Edge Detection**
https://www.youtube.com/watch?v=PS7zHKwXWRM











# Canny Edge Detector

1. Filter image with derivative of Gaussian

   $\sigma$ : width of the Gaussian blur

2. Find magnitude and orientation of gradient

3. Non-maximum suppression

   to thin the edges detected in an image. It helps to reduce the number of redundant edge points, leaving only the most likely edge candidates.

4. Linking and thresholding (hysteresis):
   - Define two thresholds: low and high
   - Use the high threshold to **start edge curves** and the low threshold to **continue them**

# Canny Edge Detector: Example



original



Canny with $\sigma = 1$



Canny with $\sigma = 2$

- The choice of $\sigma$ depends on desired behavior
  - large $\sigma$ detects "large-scale" edges
  - small $\sigma$ detects fine edges

# The End